

11__overfitting__tests

July 4, 2026

1 11 — Overfitting tests (did we find an edge, or make our own luck?)

Before spending weeks paper-trading, we stress-test the backtest edge with three in-sample statistical tests. They can't fully replace forward data, but if any comes back ugly, we've saved ourselves the weeks.

We run them on each position's **exit-inclusive realized return** (`realizedPnl / totalBought`) — the honest “perfect-mirror” signal — for the whole candidate pool, all from cache.

1. **Permutation test (consensus)**: is the “more backers → better” effect real, or could shuffling produce it by chance?
2. **Random-roster test (skill selection)**: does copying our *qualified* traders beat copying *random* traders from the same pool?
3. **PBO — Probability of Backtest Overfitting (López de Prado CSCV)**: across the qualification-gate settings we *could* have chosen, how often does the in-sample best underperform out-of-sample? High PBO = we likely overfit.

```
[1]: import importlib, pmc
importlib.reload(pmc)
from pmc import CFG, get_leaderboard, get_closed_positions
import pandas as pd, numpy as np
rng = np.random.default_rng(42)
```

1.1 0. Rebuild the candidate universe (cached) with realized returns

```
[2]: cands = {}
for w in ("ALL", "MONTH"):
    for r in get_leaderboard(window=w, limit=CFG.WF_CANDIDATES):
        cands.setdefault(r["wallet"], r)
rows = []
for wallet in cands:
    for p in get_closed_positions(wallet, max_positions=600):
        cost = float(p.get("totalBought") or 0)
        if cost <= 0:
            continue
        rows.append({"wallet": wallet, "conditionId": p.get("conditionId"),
            ↪ "outcome": p.get("outcome"),
```

```

        "entry_ts": int(p.get("timestamp") or 0), "endDate": p.
        ↪get("endDate"),
        "realizedPnl": float(p.get("realizedPnl") or 0), "cost":_
        ↪cost,
        "won": 1 if float(p.get("realizedPnl") or 0) > 0 else 0})
h = pd.DataFrame(rows)
h["res_ts"] = (pd.to_datetime(h["endDate"], errors="coerce", utc=True).
        ↪astype("int64") // 10**9)
h = h[(h["entry_ts"] > 0) & (h["res_ts"] > 0) & (h["res_ts"] > h["entry_ts"])]
        ↪copy()
h["roi"] = (h["realizedPnl"] / h["cost"]).clip(-1, 10)      # exit-inclusive_
        ↪realized return, capped
h = h.sort_values("entry_ts").reset_index(drop=True)
print(f"{len(h)} candidate positions | {h['wallet'].nunique()} wallets")

def pit_qmask(df, min_trades, min_wr):
    m = np.zeros(len(df), dtype=bool)
    for wal, idx in df.groupby("wallet").groups.items():
        w = df.loc[idx]; rs = w.sort_values("res_ts")
        rt = rs["res_ts"].values; wc = np.cumsum(rs["won"].values)
        k = np.searchsorted(rt, w["entry_ts"].values, side="left")
        wins = np.where(k > 0, wc[np.clip(k - 1, 0, len(wc) - 1)], 0)
        wr = np.where(k > 0, wins / np.maximum(k, 1), 0.0)
        m[np.array(idx)] = (k >= min_trades) & (wr >= min_wr)
    return m

# main strategy gate (same as nb 07-10)
h["qual"] = pit_qmask(h, CFG.WF_MIN_TRAILING_TRADES, CFG.
        ↪WF_MIN_TRAILING_WINRATE)
q = h[h["qual"]].copy()
q["backers"] = q.groupby(["conditionId", "outcome"])["wallet"].
        ↪transform("nunique")
print(f"qualified positions: {len(q)} | median realized ROI: {q['roi'].median():
        ↪.3f}")

```

19745 candidate positions | 212 wallets
qualified positions: 5067 | median realized ROI: 0.063

1.2 1. Permutation test — is the consensus effect real?

Observed effect = median ROI(backers 2) – median ROI(backers 1). We shuffle the **backers** labels across qualified positions many times to build the null; the p-value is the share of shuffles whose gap is the real one.

```

[3]: bk = q["backers"].values
     roi = q["roi"].values
     def gap(bk_arr):

```

```

hi = roi[bk_arr >= 2]; lo = roi[bk_arr >= 1]
if len(hi) == 0 or len(lo) == 0:
    return np.nan
return np.median(hi) - np.median(lo)

obs = gap(bk)
R = 2000
null = np.array([gap(rng.permutation(bk)) for _ in range(R)])
p_val = (np.sum(null >= obs) + 1) / (R + 1)
print(f"observed consensus gap (median ROI 2 minus 1): {obs:+.3f}")
print(f"permutation null: mean {np.nanmean(null):+.3f}, 95th pct {np.
↳nanpercentile(null,95):+.3f}")
print(f"p-value: {p_val:.4f} -> {'SIGNIFICANT' if p_val < 0.05 else 'NOT_
↳significant'}")

```

```

observed consensus gap (median ROI 2 minus 1): +0.122
permutation null: mean +0.000, 95th pct +0.014
p-value: 0.0005 -> SIGNIFICANT

```

1.3 2. Random-roster test — does skill selection add value?

Compare the qualified roster's median ROI to that of random rosters of the same size drawn from the candidate pool. NB: both draw from *today's leaderboard*, so this tests whether our **skill filter** beats random membership — it does not remove survivorship (nothing in-sample can).

```

[4]: q_wallets = q["wallet"].unique()
k_wallets = len(q_wallets)
all_wallets = h["wallet"].unique()
obs_med = q["roi"].median()

by_wallet = {w: h.loc[idx, "roi"].values for w, idx in h.groupby("wallet").
↳groups.items()}
def roster_median(ws):
    v = np.concatenate([by_wallet[w] for w in ws]) if len(ws) else np.array([])
    return np.median(v) if len(v) else np.nan

R = 2000
null = np.array([roster_median(rng.choice(all_wallets, size=k_wallets,
↳replace=False)) for _ in range(R)])
p_val = (np.sum(null >= obs_med) + 1) / (R + 1)
print(f"qualified roster median ROI: {obs_med:+.3f} ({k_wallets} wallets)")
print(f"random-roster null: mean {np.nanmean(null):+.3f}, 95th pct {np.
↳nanpercentile(null,95):+.3f}")
print(f"p-value: {p_val:.4f} -> {'skill filter ADDS value' if p_val < 0.05_
↳else 'NO evidence skill filter helps'}")

```

```

qualified roster median ROI: +0.063 (66 wallets)
random-roster null: mean +0.023, 95th pct +0.050

```

p-value: 0.0200 -> skill filter ADDS value

1.4 3. PBO — Probability of Backtest Overfitting (CSCV)

Each qualification-gate setting we *could* have picked is a “trial”. We build a matrix of per-position returns (return = realized ROI if the wallet qualifies under that gate at that position, else 0 = not traded), then run combinatorially-symmetric cross-validation: repeatedly split positions into in-sample / out-of-sample halves, pick the best trial IS, and see how it ranks OOS. **PBO = fraction of splits where the IS-best lands below the OOS median.** Rule of thumb: PBO < 0.5 is reassuring; well below is good; near/above 0.5 is a red flag that our ‘best’ is likely luck.

```
[5]: from itertools import combinations
# trials = grid of gate settings we might have chosen
grid = [(mt, wr) for mt in (20, 30, 40, 50) for wr in (0.50, 0.55, 0.60)]
cols = []
for mt, wr in grid:
    m = pit_qmask(h, mt, wr)
    ret = np.where(m, h["roi"].values, 0.0) # trade only if qualified, else 0
    # flat
    cols.append(ret)
M = np.vstack(cols).T # positions x trials
lo, hi = np.percentile(M, [1, 99])
M = np.clip(M, lo, hi) # winsorize for stable Sharpe

def sharpe(x):
    s = x.std()
    return x.mean() / s if s > 1e-9 else 0.0

S = 10 # even number of groups
groups = np.array_split(rng.permutation(len(M)), S)
lambdas = []
for combo in combinations(range(S), S // 2):
    is_idx = np.concatenate([groups[g] for g in combo])
    oos_idx = np.concatenate([groups[g] for g in range(S) if g not in combo])
    is_perf = np.array([sharpe(M[is_idx, t]) for t in range(M.shape[1])])
    oos_perf = np.array([sharpe(M[oos_idx, t]) for t in range(M.shape[1])])
    best = np.argmax(is_perf)
    # relative rank of the IS-best among OOS performances (1 = best OOS)
    rank = (oos_perf <= oos_perf[best]).mean()
    w_bar = min(max(rank, 1e-6), 1 - 1e-6)
    lambdas.append(np.log(w_bar / (1 - w_bar)))
lambdas = np.array(lambdas)
PBO = float((lambdas < 0).mean())
print(f"trials tested: {len(grid)} gate settings | CSCV splits: {len(lambdas)}")
print(f"Probability of Backtest Overfitting (PBO): {PBO:.3f}")
print(" -> " + ("LOW overfitting risk" if PBO < 0.3 else
               "MODERATE - interpret with care" if PBO < 0.5 else
               "HIGH - the 'best' gate is likely luck"))
```

trials tested: 12 gate settings | CSCV splits: 252
Probability of Backtest Overfitting (PBO): 0.000
-> LOW overfitting risk

1.5 4. Verdict & how it feeds the paper

Read the three together: - **Permutation $p < 0.05$** → the consensus effect is unlikely to be chance.
- **Random-roster $p < 0.05$** → our skill filter genuinely beats picking random leaderboard wallets (i.e. the qualification does work — though survivorship of the pool remains). - **PBO $< \sim 0.3$** → across the gate choices, we're not just cherry-picking a lucky setting.

If all three are favourable, the backtest edge is credible enough to justify the forward paper-trade (which is what removes the residual survivorship). If any is ugly — especially a high PBO — we treat the +returns as suspect and either simplify the strategy or stop.

For the LaTeX paper, these become the “robustness / overfitting” section: permutation p-values, the random-roster null histogram, and the PBO statistic (a recognised López de Prado diagnostic) are exactly what a referee will look for.

Caveats: these tests use positions as exchangeable observations and share the same survivorship-selected pool; they bound overfitting-across-variants and signal-vs-noise, not survivorship. Forward paper-trading remains the decisive test.